

Demands

There are plenty of modules that you need to write in this lab:

"Simple_Single_CPU.v", "Adder.v", "ALU.v", "ALU_Ctrl.v", "Decoder.v",
"Instr_Memory.v", "Mux2to1.v", "Mux3to1.v", "Program_Counter.v", "Reg_File.v",
"Shifter.v", "Sign_Extend.v", "Zero_Filled.v", "Data_Memory.v", "Testbench.v"

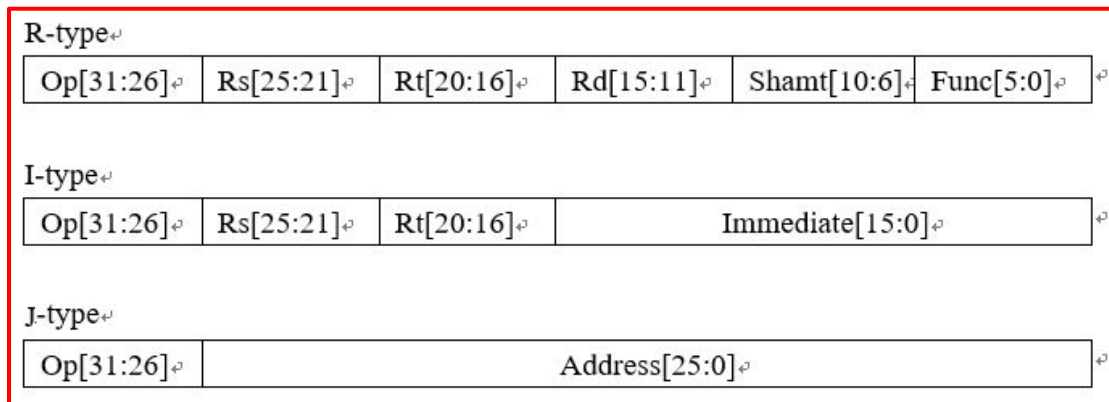
All these files are provided, please make use of them and **don't create additional .v files.**

Requirement Description

We are going to implement these instructions, and we recommend you to write them in such sequence:

- R-type: add, sub, and, or, nor, slt, sll, srl, **sllv**, **srlv**, **jr**
- I-type: addi, lw, sw, beq, bne, **blt**, **bnez**, **bgez**
- J-type: jump, **jal**

Instruction Format:



R-type Instruction set and ADDI (9% * 5)

Instruction	Example	Meaning	Op Field[31:26]	Shamt	Function Field
add	add rd, rs, rt	Reg[rd]=Reg[rs]+Reg[rt]	6'b000000	x	6'b100011
sub	sub rd, rs, rt	Reg[rd]=Reg[rs]-Reg[rt]	6'b000000	x	6'b010011
and	and rd, rs, rt	Reg[rd]=Reg[rs]&Reg[rt]	6'b000000	x	6'b011111
or	or rd, rs, rt	Reg[rd]=Reg[rs] Reg[rt]	6'b000000	x	6'b101111
nor	nor rd, rs, rt	Reg[rd]=~(Reg[rs] Reg[rt])	6'b000000	x	6'b010000
slt	slt rd, rs, rt	If (Reg[rs]<Reg[rt]) Reg[rd]=1 else Reg[rd]=0	6'b000000	x	6'b010100
sll	sll rd, rt	Reg[rd]= (unsigned) Reg[rt]<<(unsigned) amt	6'b000000	amt	6'b010010
srl	srl rd, rt	Reg[rd]= (unsigned) Reg[rt]>>(unsigned) amt	6'b000000	amt	6'b100010
addi	addi rt,rs,imm	Reg[rt]=Reg[rs] + imm	6'b010011	x	x

Advanced Set: Part 1 (10%)

- slv (Shift left logical variable) and srlv (Shift right logical variable)

Instruction	Example	Meaning	Op Field[31:26]	Shamt	Function Field
slv	slv rd, rt, rs	$\text{Reg}[\text{rd}] = (\text{unsigned}) \text{Reg}[\text{rt}] \ll (\text{unsigned}) \text{Reg}[\text{rs}]$	6'b000000	x	6'b011000
srlv	srlv rd, rt, rs	$\text{Reg}[\text{rd}] = (\text{unsigned}) \text{Reg}[\text{rt}] \gg (\text{unsigned}) \text{Reg}[\text{rs}]$	6'b000000	x	6'b101000

I-type Instruction and Jump (25%)

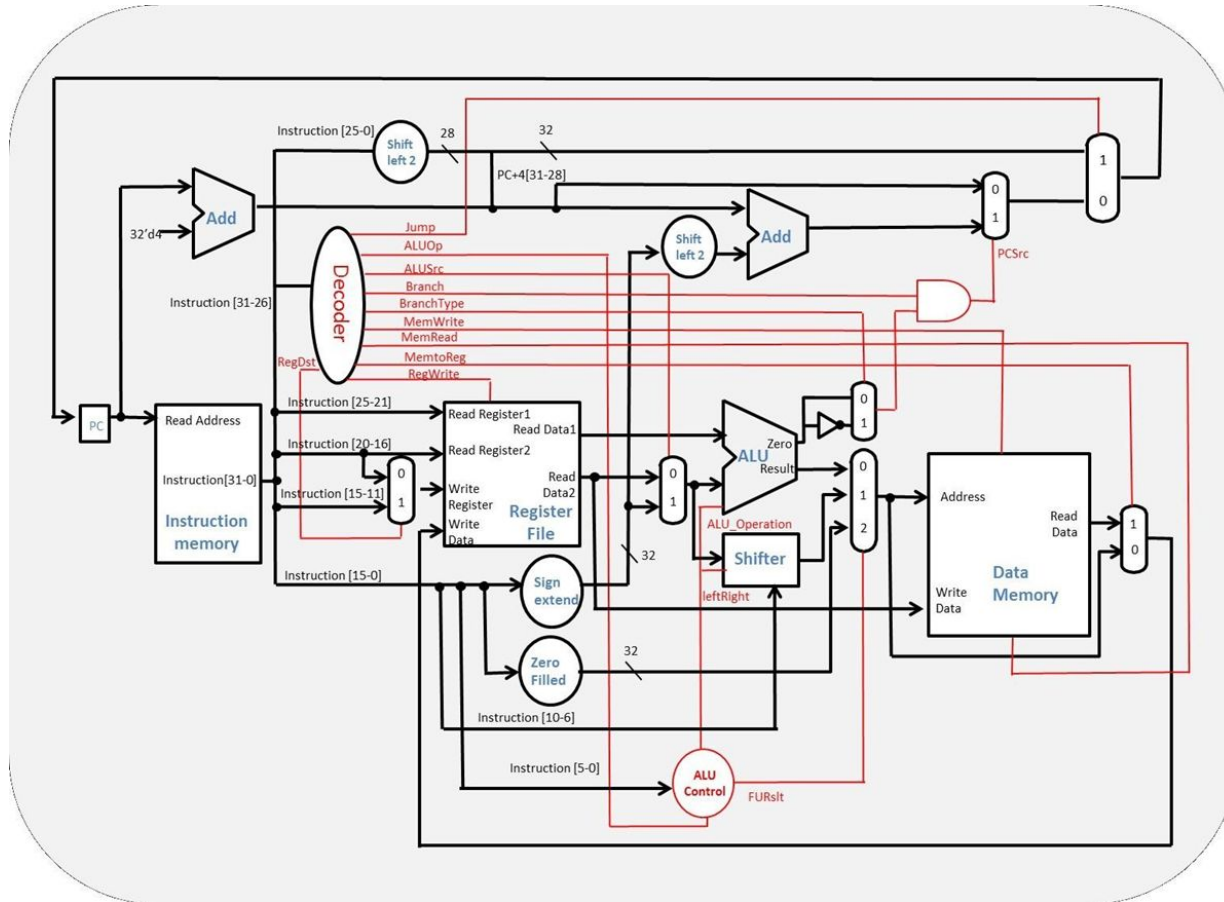
Instruction	Example	Meaning	Op Field[31:26]
lw	lw rt, imm(rs)	$\text{Reg}[\text{rt}] \leftarrow \text{Mem}[\text{Reg}[\text{rs}] + \text{imm}]$	6'b011000
sw	sw rt, imm(rs)	$\text{Mem}[\text{Reg}[\text{rs}] + \text{imm}] \leftarrow \text{Reg}[\text{rt}]$	6'b101000
beq	beq rs, rt, imm	if($\text{Reg}[\text{rs}] == \text{Reg}[\text{rt}]$) then $\text{PC} = \text{PC} + 4 + (\text{imm} \ll 2)$	6'b011001
bne	bne rs, rt, imm	if($\text{Reg}[\text{rs}] \neq \text{Reg}[\text{rt}]$) then $\text{PC} = \text{PC} + 4 + (\text{imm} \ll 2)$	6'b011010
jump	jump address	$\text{PC} = \{\text{PC}[31:28], \text{address}[25:0] \ll 2\}$	6'b001100

Advanced Set: Part 2 (10%)

- jal (Jump and link)
 - In MIPS, 31th register is used to save return address for function call Reg[31] save PC+4 and perform jump
- jr (Jump to the address in the register rs)

Instruction	Example	Meaning	Op Field[31:26]	Function Field[5:0]
jal	jal address	Reg[31]=PC+4 PC={PC[31:28],address[25:0]<<2}	6'b001111	x
jr	jr rs	PC=Reg[rs]	6'b000000	6'b000001

Architecture



- You should follow the architecture to implement your cpu (but with some tweak).